Taraju — External Integrations, Supabase & Cloud Run Setup

Oct 17, 2026 · @DeltaOS

Summary: accounts and APIs to set up

Taraju needs 11 external accounts before the Week 13 pilot go-live. Five block a sprint if they are late: MSG91 DLT (W3), a GST Suvidha Provider (W4), Supabase (W1), Google Cloud (W1) and Razorpay KYC (W10). Start DLT, GSP and Razorpay paperwork in Week 1, because each takes 1–3 weeks of approvals.

Service	What Taraju uses it for	Repo	Needed by	Lead time	Approx. cost
Supabase (Postgres)	All service databases	every Go service	W1	same day	Pro plan ~$25/mo + compute
Google Cloud	Cloud Run, Artifact Registry, Secret Manager, Pub/Sub, Scheduler, Storage	all	W1	same day (billing account)	~₹10–25k/mo (see DEPLOYMENT.md)
GitHub org	14 repos, Cloud Build triggers	all	W1	same day	free–Team plan
Domain + DNS	app.taraju.in, api.taraju.in	dukaan, dwaar	W3	1 day	~₹1k/yr
MSG91 + TRAI DLT	OTP and alert SMS	soochna	W3	1–2 weeks (DLT)	per-SMS
Gupshup (WhatsApp BSP)	OTP fallback, bills, receipts, alerts	soochna	W3 (OTP), W10 (alerts)	1–3 weeks (Meta verification)	per-message
GST Suvidha Provider (GSP)	Live GSTIN lookup, e-Way bill, e-Invoice IRN, later GSTR filing	kar	W4 (GSTIN), W8 (e-Way/e-Invoice)	1–2 weeks (contract)	per-call or annual
Redis (Memorystore or Upstash)	OTP, rate limits, sessions, KPI counters	dwaar, pehchaan, kar, buddhi	W3	same day	~$10–40/mo

Service	What Taraju uses it for	Repo	Needed by	Lead time	Approx. cost
Razorpay	UPI QR, payment links, refunds, webhooks	lenden	W10 (sandbox), W12 (live)	3–10 days (KYC)	~2% per transaction
Anthropic Claude API	Bill scanning, ask-your-books, weekly summary	buddhi	W12 (pilot)	same day	per token, capped per tenant
Sentry	Frontend error tracking	dukaan	W12	same day	free tier

Costs are approximate list prices as of this writing; confirm on each vendor's pricing page before budgeting.

GST integrations (kar)

All government GST APIs are reached through one **GST Suvidha Provider (GSP)** contract; sign it by Week 3 so live GSTIN lookup works in Week 4. Until then kar runs with `GSTIN_PROVIDER=checksum`, which validates format, checksum and state code offline but returns no legal name or status.

API	What it gives Taraju	Who registers	Needed by	Status in code
GSTIN taxpayer search	Legal name, trade name, address, Active/Cancelled status when a party is created	Taraju (one GSP account for all tenants)	W4	Built: `GSTIN_PROVIDER=gsp`, cached 7 days
e-Way Bill (NIC)	e-Way bill number for consignments above ₹50,000	Each tenant creates an API user on the e-Way Bill portal and links it to the GSP	W8	JSON payload built; portal submission not built
e-Invoice (IRP)	IRN + signed QR for B2B invoices (mandatory above the turnover threshold)	Each tenant creates an API user on the e-Invoice portal and links it to the GSP	W8	INV-01 JSON built; IRN submission not built

API	What it gives Taraju	Who registers	Needed by	Status in code
GSTR-1 / GSTR-3B filing	Push returns to the GST portal	Each tenant authorises via OTP from the GST portal	Phase 3	JSON + CSV export built; filing not built

Choosing a GSP

Shortlist Masters India, ClearTax, Cygnet, IRIS GST and Vayana. Ask each for: sandbox access, per-call price for taxpayer search, e-Way and e-Invoice bundles, an API that lets one Taraju account act for many client GSTINs, and SLA. Pick one that serves **both** e-Way bill and e-Invoice, so tenants link one provider.

Steps

1. Sign the GSP contract and get sandbox credentials (client ID, client secret, base URL).
2. Put them in Secret Manager as `gsp-client-id` and `gsp-client-secret`; set `GSP_BASE_URL` and, if the GSP's path differs, `GSP_SEARCH_PATH` on kar.
3. Set `GSTIN_PROVIDER=gsp` on kar in dev and test a party create with `?verify=true` in bahi.
4. Before W8: write a one-page guide for store owners on creating the e-Way Bill and e-Invoice API users (portal → Registration → For GSP → select your GSP), and add a screen in dukaan settings to store each tenant's API username/password.
5. Build the submission calls (IRN generation, e-Way bill generation and cancel) in kar against the sandbox; store `irn`, `ewb_no` in the columns that already exist.

Open point: onboarding does not yet ask for the store's address and PIN code, and the IRP rejects e-Invoices without them. Add both to onboarding before W8.

Messaging: SMS and WhatsApp (soochna)

OTP login in Week 3 depends on one approved SMS template on MSG91, and TRAI DLT approval is the slowest step, so apply in Week 1. WhatsApp via Gupshup can follow by Week 10 for bills and alerts; until then soochna's `console` provider logs messages instead of sending them.

SMS — MSG91 + TRAI DLT

1. Register Confidus Technologies as a Principal Entity on a DLT portal (Jio, Airtel, Vodafone Idea or BSNL). Needs PAN, GST certificate and a letter of authorisation.

2. Register a 6-letter sender header, for example `TARAJU`.
3. Register content templates. Each variable in the template must match soochna's positional order:

soochna template	Variables, in order
`otp`	otp, minutes
`invoice`	party_name, business, number, amount, date
`payment_receipt`	party_name, business, amount, ref
`ledger_statement`	party_name, business, balance, as_of
`stock_low`	item, on_hand, reorder_level, business
`batch_expiring`	item, batch, expiry, days, business
`import_done`	kind, imported, total, failed
`weekly_summary`	text

4. Create an MSG91 account, link the DLT entity and header, and create one Flow per template with its DLT template ID.
5. Store the auth key as secret `msg91-auth-key`; set `SMS_PROVIDER=msg91` and `MSG91_TEMPLATE_<NAME>` (the Flow IDs) on soochna.

WhatsApp — Gupshup

1. Verify the business in Meta Business Manager (same legal documents as DLT).
2. Create a Gupshup WhatsApp app and attach a dedicated phone number that is not on any WhatsApp app.
3. Submit one message template per row above. Use the Authentication category for `otp` and Utility for the rest; marketing templates cost more and are not needed.
4. Store the API key as secret `gupshup-api-key`; set `WHATSAPP_PROVIDER=gupshup`, `GUPSHUP_SOURCE` (the number, `91XXXXXXXXXX`), `GUPSHUP_APP_NAME` and `GUPSHUP_TEMPLATE_<NAME>`.
5. In Gupshup and MSG91, point delivery reports to `https://api.taraju.in/api/soochna/v1/webhooks/gupshup?token=<WEBHOOK_TOKEN>` (and `/msg91`). The gateway already lets these through without a login.

Template IDs are set once with `gcloud run services update soochna --update-env-vars=...`; deploys keep them.

Payments: Razorpay (lenden)

lenden is built against **one** Razorpay account, so today every customer payment would settle into Taraju's bank account, not the store's. Decide the money flow before live keys go in (Week 12); sandbox work in Week 10 is unaffected.

Option	Money lands in	Work in lenden	Fit
A. Each store brings its own Razorpay keys	Store's account directly	Store keys per tenant (encrypted), pick keys by tenant, one webhook secret per tenant	Recommended for the pilot: simplest compliance, no money handling by Taraju
B. Razorpay Partner programme (sub-merchants / OAuth)	Store's account	OAuth onboarding flow, act on behalf of sub-merchant	Best at scale; needs partner approval
C. Taraju collects, then pays out (Route)	Taraju first, then store	Transfers after capture	Avoid: makes Taraju a payment aggregator under RBI rules

Steps (sandbox, Week 10)

1. Create a Razorpay account and complete KYC (business PAN, GST, bank account, website with pricing, refund and privacy pages).
2. In test mode, generate API keys. Store as secrets `razorpay-key-id`, `razorpay-key-secret`.
3. Ask Razorpay support to enable **QR Codes** (UPI) and **Payment Links** on the account; QR codes are not on by default.
4. Dashboard → Webhooks: URL `https://api.<env>.taraju.in/api/lenden/v1/webhooks/razorpay`, a strong secret (store as `razorpay-webhook-secret`), events `payment.captured`, `qr_code.credited`, `payment_link.paid`, `refund.processed`.
5. Set `PAYMENT_GATEWAY=razorpay` and `PAYEE_DISPLAY_NAME` on lenden; leave `mock` locally. lenden refuses `mock` in prod.
6. Test: POS → UPI QR → pay with a test UPI ID → invoice shows paid and bahi has a receipt voucher.

Steps (live, Week 12)

1. Implement the chosen option (A means per-tenant keys in lenden plus a settings screen in dukaan).
2. Swap to live keys and a live webhook secret in the prod project only.
3. Keep the 15-minute `lenden-reconcile` scheduler job on; it catches payments whose webhook was missed.

AI and other services

With the database on Supabase, use **Upstash Redis** instead of Memorystore: it needs no VPC, so Cloud Run services can drop their VPC settings entirely. Everything else here is a same-day sign-up.

Anthropic Claude API (buddhi)

1. Create an organisation at console.anthropic.com, add billing, and create a workspace per environment (dev, prod).
2. Set a monthly spend limit on each workspace; buddhi also enforces `AI_MONTHLY_TOKEN_BUDGET` per tenant (2M tokens default).
3. Create an API key per workspace; store as secret `anthropic-api-key`.
4. Models are set by env: `AI_MODEL_SMART` and `AI_MODEL_FAST`, both `claude-opus-5-5` today. Try `claude-haiku-5-5` for bill scanning only after measuring accuracy on 50+ real supplier bills.
5. AI is off per tenant until `ai_enabled` is set (`PUT /v1/ai/settings`); turn it on for pilot stores only.

Redis

Choice	Network	URL format	When
Upstash (region ap-south-1, Mumbai)	Public, TLS	`rediss://default:<password>@<host>:6379`	Recommended with Supabase
Memorystore (asia-south1)	VPC only	`redis://10.x.x.x:6379/0`	Only if you keep the VPC for other reasons

Store the URL as secret `redis-url`. Used by dwaar (rate limits), pehchaan (OTP, refresh tokens), kar (optional GSTIN cache) and buddhi (live KPIs).

Sentry, domain, GitHub

- **Sentry:** create a Next.js project; set `NEXT_PUBLIC_SENTRY_DSN` as a dukaan build argument. The SDK is not wired in yet.
- **Domain:** buy `taraju.in` (or chosen name). Plan `app.<env>.taraju.in` → dukaan and `api.<env>.taraju.in` → dwaar.
- **GitHub:** create the org, push the 14 repos, and add a fine-grained token with read access to `mool` as secret `github-token` (Cloud Build clones `mool` with it).

Database setup on Supabase

Use one Supabase project per environment in the Mumbai region, with **one schema and one login role per service** inside the `postgres` database. No service code changes: each role's `search_path` points at its schema, and migrations run automatically at startup. Two new files in `sanchalan` do the work: `supabase/setup.sql` and `scripts/supabase-secrets.sh`.

Steps (repeat for dev, staging, prod)

1. **Create the project.** Name `taraju-<env>`, region **South Asia (Mumbai)** — same city as Cloud Run `asia-south1`, so queries stay ~1–3 ms. Pro plan for staging and prod; save the database password in a password manager.
2. **Create schemas and roles.** Dashboard → SQL Editor → paste `sanchalan/supabase/setup.sql` → Run. It creates schemas `pehchaan`, `bahi`, `kar`, `aadesh`, `lenden`, `soochna`, `aayaat`, `bhandar`, `nirmaan`, `buddhi`, each owned by `<service>_svc`, and returns a password per new role. Copy the result into a local `creds.csv` (service,role,password).
3. **Find the connection details.** Dashboard → Connect → **Session pooler**: note the host (e.g. `aws-0-ap-south-1.pooler.supabase.com`). Project Settings → General: note the Reference ID.
4. **Store connection strings.** Run:

```
cd sanchalan
ENV=dev SUPABASE_REF=<ref> SUPABASE_POOLER_HOST=<host> ./scripts/supabase-secrets.sh creds.csv
rm -P creds.csv
```

This writes `<service>-database-url` secrets in Google Secret Manager, which the Cloud Run deploys already read.

5. **Lock down the Data API.** Project Settings → API: keep exposed schemas to `public` only (the service schemas are never exposed), or turn the Data API off; Taraju does not use it. Setup.sql already revokes `anon` and `authenticated` from every service schema.
6. **Enforce SSL.** Database settings → SSL enforcement on. The connection strings use `sslmode=require`.
7. **Backups.** Pro includes daily backups; add Point-in-Time Recovery on **prod** before the Week 12 pilot. Run a restore drill into a scratch project in Week 26.
8. **Deploy a service** (Cloud Run section below). On first start it creates its tables and `schema_migrations` in its own schema; check Table Editor → schema dropdown.

Connection string format

```
postgres://aadesh_svc.<project-ref>:<password>@<session-pooler-host>:5432/postgres?sslmode=require&pool_max_conns=5
```

Rule	Why
Use the **session** pooler, port 5432	The migration runner holds a session advisory lock and pgx uses prepared statements; the transaction pooler (port 6543) breaks both
Not the direct `db.<ref>.supabase.co` host	It is IPv6-only unless you buy the IPv4 add-on; Cloud Run egress is IPv4
`pool_max_conns=5` per instance	Session mode holds one Postgres connection per client connection; 11 services × instances × 5 must stay under your compute's max connections (Dashboard → Database settings)
Role `CONNECTION LIMIT 40`	One runaway service cannot starve the others

Raise compute size before raising `pool_max_conns` when the pilot grows. Each role also has `statement_timeout = 30s`.

Local development

No change: `sanchalan/docker-compose.yml` runs Postgres 16 with one database per service, and each repo's `.env` points at it. Supabase is only for dev, staging and prod in the cloud.

Cloud Run deploy setup

A fresh environment takes about half a day: bootstrap Google Cloud, load Supabase secrets, deploy 12 Cloud Run services in dependency order, then wire IAM, Pub/Sub and Scheduler. Every step is a script in `sanchalan/scripts`; each repo's `cloudbuild.yaml` does the build and deploy.

One-time, per environment

1. **Project and billing.** Create `taraju-<env>` in Google Cloud, link billing, install `gcloud` and run `gcloud auth login`.
2. **Bootstrap.** `ENV=dev ./scripts/gcp-bootstrap.sh` — enables APIs, creates Artifact Registry `taraju`, the VPC used by Cloud Run egress, bucket `taraju-<env>-files`, service accounts per service, and placeholder secrets. It skips Cloud SQL and Memorystore by default (`DB_PROVIDER=supabase`, `REDIS_PROVIDER=upstash`).
3. **Database secrets.** Supabase steps above → `<service>-database-url` secrets.
4. **Fill the placeholder secrets** (`CHANGE_ME`): `redis-url`, `github-token`, and the vendor keys as each vendor goes live:

```
printf '%s' 'rediss://default:...@...upstash.io:6379' | gcloud secrets
versions add redis-url --data-file=-
```

5. **Connect GitHub.** Cloud Build → Repositories (2nd gen) → connect the GitHub org, then `ENV=dev ./scripts/triggers-setup.sh` (push to `main` → dev; tag `v*` → staging).
6. **Deploy in order.** `ENV=dev ./scripts/deploy.sh all` runs each repo's Cloud Build in this order: soochna → pehchaan → dwaar → kar → bahi → bhandar → aadesh → lenden → aayaat → nirmaan → buddhi → dukaan.
7. **Who may call whom.** `ENV=dev ./scripts/iam-invokers.sh` grants `run.invoker` (gateway → every service; service → service per the architecture doc; Pub/Sub and Scheduler → services).
8. **Events.** `ENV=dev ./scripts/pubsub-setup.sh` creates 10 topics, push subscriptions to each service's `/events`, and a dead-letter topic.
9. **Schedules.** `ENV=dev ./scripts/scheduler-setup.sh` creates 6 jobs (soochna retry, lenden reconcile, bhandar reorder and expiry scans, buddhi forecast and weekly summary).
10. **Domains.** Create an external HTTPS load balancer with managed certificates: `api.<env>.taraju.in` → dwaar, `app.<env>.taraju.in` → dukaan, plus a Cloud Armor policy. Then redeploy dwaar with `_INGRESS=internal-and-cloud-load-balancing` so the raw `run.app` URL cannot bypass Cloud Armor.

What each service runs with

Service	Ingress	Min instances	Why
dwaar, dukaan	load balancer only (public)	1	Every request enters here
pehchaan, aadesh, bahi, bhandar, lenden	internal	1	Billing and login must not cold-start
soochna, kar, aayaat, nirmaan, buddhi	internal	0	Async or occasional

Internal services require IAM auth: callers send a Google ID token in `X-Serverless-Authorization`, which `mool/svc` and dwaar already do. Publishing services run with `--no-cpu-throttling` so the outbox keeps flushing between requests. aayaat also deploys a Cloud Run **Job**, `aayaat-worker`, for imports above 50,000 rows.

Verify after a deploy

- [ ] `curl https://api.<env>.taraju.in/healthz` returns `ok`; `GET /api` lists all 10 services as configured
- [ ] OTP login works from dukaan (SMS arrives, or `console` log in soochna for dev)
- [ ] A cash sale shows in bahi's day book and reduces stock in bhandar within seconds (events flowing)
- [ ] Pub/Sub dead-letter subscription has 0 messages
- [ ] Cloud Logging shows no `severity=ERROR` for the new revision

Rollback

Every image is tagged with the git SHA and Cloud Run keeps old revisions:

```
gcloud run services update-traffic aadesh --region=asia-south1 --to-revisions=<previous-revision>=100
```

Migrations only add tables and columns, so the previous revision still runs against the newer schema.

Secrets and environment variables per service

Every Go service gets `jwt-secret` and `service-secret` (identical across services) plus the shared settings `APP_ENV`, `EVENTS_DRIVER=pubsub`, `GCP_PROJECT`, `EVENTS_TRUST_IAM=true`, `USE_ID_TOKENS=true`, `AUTO_MIGRATE=true`. The table lists only what is extra. Local values are in each repo's `.env.example`.

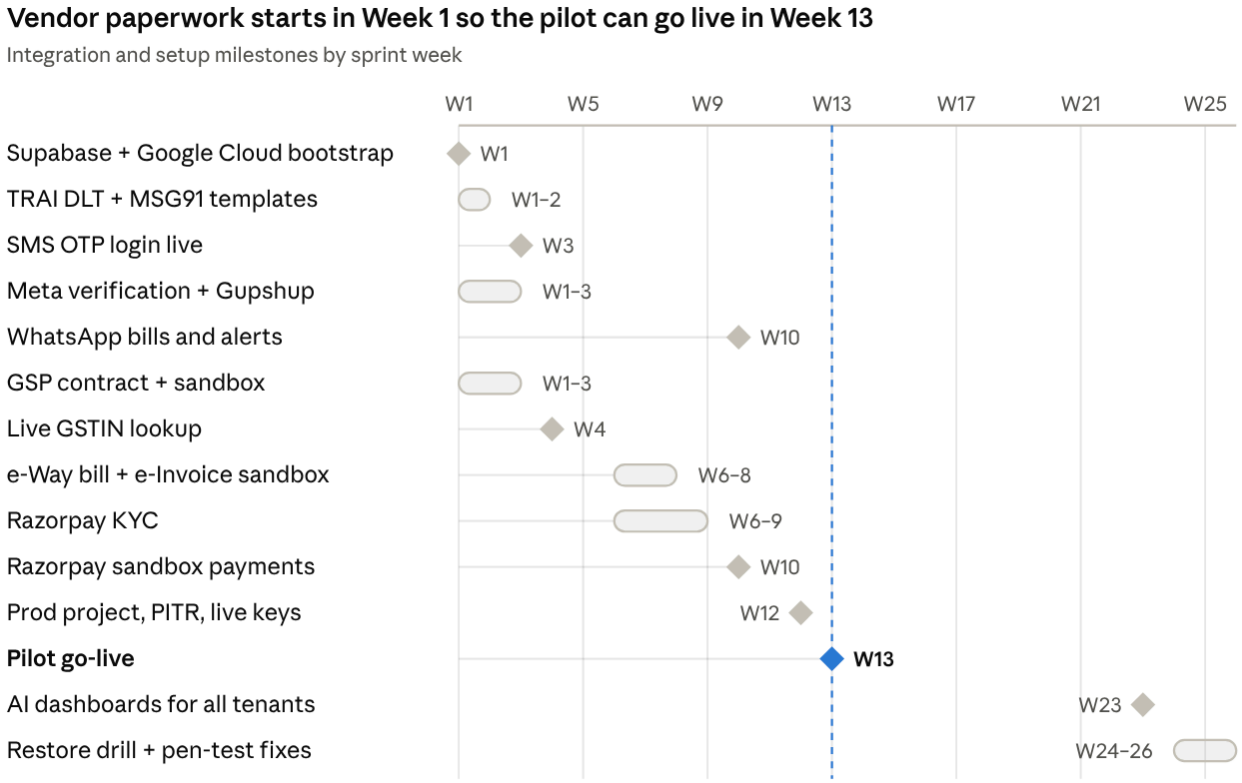
Service	Secrets (Secret Manager)	Extra env vars
dwaar	`redis-url`	`PEHCHAAN_URL` … `BUDDHI_URL` (10 services), `RATE_LIMIT_PUBLIC_PER_MIN`, `RATE_LIMIT_USER_PER_MIN`, `CORS_ORIGINS`
pehchaan	`pehchaan-database-url`, `redis-url`	`SOOCHNA_URL`
soochna	`soochna-database-url`, `msg91-auth-key`, `gupshup-api-key`	`SMS_PROVIDER=msg91`, `WHATSAPP_PROVIDER=gupshup`, `GUPSHUP_SOURCE`, `GUPSHUP_APP_NAME`, `MSG91_TEMPLATE_*`, `GUPSHUP_TEMPLATE_*`, `WEBHOOK_TOKEN`, `PEHCHAAN_URL`
kar	`kar-database-url`, `gsp-client-id`, `gsp-client-secret`, `redis-url` (optional)	`GSTIN_PROVIDER=gsp`, `GSP_BASE_URL`, `GSP_SEARCH_PATH`, `AADESH_URL`, `PEHCHAAN_URL`, `BAHI_URL`
bahi	`bahi-database-url`	`KAR_URL`
bhandar	`bhandar-database-url`	`EXPIRY_ALERT_DAYS`
aadesh	`aadesh-database-url`	`PEHCHAAN_URL`, `BAHI_URL`
lenden	`lenden-database-url`, `razorpay-key-id`, `razorpay-key-secret`, `razorpay-webhook-secret`	`PAYMENT_GATEWAY=razorpay`, `PAYEE_DISPLAY_NAME`
aayaat	`aayaat-database-url`	`BAHI_URL`, `BHANDAR_URL`, `STORAGE_DRIVER=gcs`, `GCS_BUCKET`, `LARGE_IMPORT_ROWS`, `WORKER_JOB_NAME`
nirmaan	`nirmaan-database-url`	`BHANDAR_URL`, `BAHI_URL`
buddhi	`buddhi-database-url`, `anthropic-api-key`, `redis-url`	`AI_MODEL_SMART`, `AI_MODEL_FAST`, `AI_MONTHLY_TOKEN_BUDGET`, `AI_REQUIRE_TENANT_FLAG`, `BAHI_URL`, `BHANDAR_URL`, `AADESH_URL`, `SOOCHNA_URL`, `PEHCHAAN_URL`

Service	Secrets (Secret Manager)	Extra env vars
dukaan	none (build args only)	`NEXT_PUBLIC_API_URL=https://api.<env>.taraju.in/api`, `NEXT_PUBLIC_SENTRY_DSN`

Service URLs on Cloud Run are fixed: `https://<service>-<project-number>.asia-south1.run.app`; the `cloudbuild.yaml` files already fill them in. Platform secret `github-token` is read by Cloud Build only.

Timeline and go-live checklist

The three approval-bound items (DLT, GSP, Razorpay KYC) start in Week 1 or soon after; everything else is a same-week task.



setup milestones · weeks 1–26

The dashed line is the Week 13 pilot go-live; nothing to its right blocks the pilot.

Before pilot go-live (Week 12)

- [ ] Prod Supabase project on Pro with Point-in-Time Recovery
- [ ] Prod secrets hold live keys: MSG91, Gupshup, GSP, Razorpay, Anthropic
- [ ] Razorpay money flow decided and built (per-store keys recommended)

- [ ] Onboarding collects store address and PIN code
- [ ] Refresh token moved from localStorage to an httpOnly cookie in dukaan
- [ ] HSN starter rates checked against the latest CBIC notification
- [ ] Load balancer, managed certs and Cloud Armor in front of dwaar and dukaan; dwaar ingress set to load balancer only
- [ ] Alerts: 5xx rate, Pub/Sub dead-letter count, Supabase connections near limit
- [ ] Pilot data imported with aayaat and opening balances reconciled

Open decisions and risks

The Razorpay money flow is the only decision that changes code before the pilot; the rest are vendor choices or schedule risks.

Item	Type	Owner	Decide by	Recommendation / fallback
Where customer payments settle	Decision	Founders	W10	Per-store Razorpay keys (option A); Partner programme later
Which GSP	Decision	Founders	W3	One that covers GSTIN search, e-Way and e-Invoice with multi-GSTIN access
Redis: Upstash vs Memorystore	Decision	Integrations intern	W2	Upstash (no VPC needed with Supabase)
DLT approval slips past W3	Risk	Integrations intern	W2	WhatsApp OTP or `console` OTP in dev; keep the pilot on WhatsApp if SMS is late
Meta business verification rejected	Risk	Founders	W3	Re-submit with GST certificate and matching website; SMS-only alerts meanwhile
Supabase connection limit under load	Risk	Backend intern	W24 load test	Raise compute size; keep `pool_max_conns=5`; cap Cloud Run max instances

Item	Type	Owner	Decide by	Recommendation / fallback
Government portal submission (IRN, e-Way number) not built	Gap	Backend intern	W8	Tenants upload kar's JSON to the portal by hand until the GSP calls are built
AI cost overrun	Risk	Founders	W12	Per-tenant token budget (2M/month) plus a workspace spend limit in the Anthropic console

Files referenced in this doc live in the `sanchalan` repo: `supabase/setup.sql`, `scripts/supabase-secrets.sh`, `scripts/gcp-bootstrap.sh`, `docs/ARCHITECTURE.md`, `docs/DEPLOYMENT.md`.